Parte 5B: Generación de casos de prueba con IA

API: Swagger Petstore (`https://petstore.swagger.io/v2`) **Fecha:** 15 de octubre de 2026

---

1. Resumen

Usé un LLM para derivar los casos de prueba de todos los endpoints de la Petstore a partir de su especificación Swagger. El modelo generó **145 casos** y **30 ambigüedades** de la spec. En lugar de aceptar la salida tal cual, **contrasté una muestra de 34 casos contra la API real**, clasifiqué los 145 con criterios explícitos y **automaticé 12 casos** dentro del framework de la Parte 1.

Resultado de la evaluación:

Clasificación	Casos
Automatizar	40
Automatizar con ajuste	18
Hallazgo (defecto o desvío de la spec)	46 (18 verificados)
Pregunta al negocio	33
Descartar	8
Total	**145**

La conclusión principal: **la IA genera casos razonables y detecta muy bien los vacíos de la spec, pero no puede decidir cuál es el comportamiento correcto.** Todos los supuestos de "respuesta exitosa" se cumplieron, mientras que ninguno de los supuestos sobre validación o autenticación se cumplió en la API real. Esa diferencia solo apareció al ejecutar.

---

2. Herramientas y enfoque

Herramienta	Uso
Claude Code (extensión de VS Code), modelo **[COMPLETAR]**	Generación de casos, scripts de verificación y clasificación, y generación de tests
Claude (chat)	Diseño de los prompts y revisión crítica de cada salida
Node.js + tsx + TypeScript	Ejecución de los scripts de verificación contra la API
Playwright	Framework donde se integraron los tests generados (Parte 1)

Decisión clave: dos contextos distintos según la tarea.

- **Para generar casos**, el agente tuvo prohibido leer el repositorio y solo podía usar `swagger.json`. El enunciado pide derivar los casos "a partir de la documentación", y si el modelo hubiera leído el código de la Parte 1, no se podría distinguir qué casos salen de la spec y cuáles de mi implementación.
- **Para automatizar**, en cambio, el agente leyó el framework de la Parte 1 a propósito, para que el código generado reutilizara sus clients, fixtures y factories, y siguiera sus convenciones.

Restricciones de la API pública. La Petstore es compartida por muchos usuarios y no aísla datos. Todos los prompts exigieron ids únicos, ejecución secuencial y no modificar datos ajenos.

---

3. Proceso

El trabajo se hizo en cinco iteraciones. Cada una partió de algo observado en la anterior.

Iteración 1: generación de casos (prompt 01)

Le entregué al modelo la spec real (`inputs/swagger.json`) en vez de pedirle que "recordara" la Petstore, y ejecuté el prompt **una vez por tag** (`pet` , `store` , `user`). Con todos los endpoints juntos, la respuesta tiende a cortarse o a perder profundidad en los últimos.

Decisiones del prompt:

- **Rol y contexto real** (API pública, compartida, a automatizar con Playwright), para que propusiera casos ejecutables.
- **Categorías explícitas** (positivo, negativo, borde, seguridad). Sin ellas, los LLM tienden a generar casi solo happy paths.
- **Columna "Supuesto" obligatoria** cuando el resultado esperado no está documentado. Fue la herramienta principal de la evaluación posterior, porque separa lo que dice la spec de lo que el modelo infiere.
- **Formato CSV fijo**, para importar la salida a una planilla y agregarle columnas de evaluación.
- **Lista de ambigüedades** de la spec, además de los casos.

Resultado: 145 casos (69 pet, 35 store, 41 user), todos con la estructura pedida, y 30 ambigüedades (`outputs/casos-*-raw.csv` y `outputs/ambiguedades-*.md`).

Primera lectura crítica:

- La mayoría de los casos dependía de un supuesto, porque la spec casi no documenta respuestas exitosas, ni 401 ni 403.
- **STO-006 contradecía su propio contexto:** usaba un id fijo (`7`) para una orden, aunque el prompt indicaba que la API es compartida. El modelo priorizó una regla de la spec (GET de órdenes solo acepta ids del 1 al 10) por sobre el contexto.
- Todos los casos de seguridad asumían 401, pero en la Parte 1 ya había creado mascotas sin token.
- **Acierto destacable:** PET-010 advirtió que JavaScript pierde precisión con ids `int64` grandes, el mismo problema que encontré en el escenario B de la Parte 1.

Iteración 2: verificación contra la API real (prompt 02)

En lugar de opinar sobre qué casos sirven, le pedí al agente un script que ejecutara una **muestra de 25 casos**, al menos uno por cada tipo de supuesto, sin credenciales, y comparara el resultado esperado con el real.

La primera versión del prompt no contemplaba las dependencias del script. La **versión 2** agregó una sección de prerrequisitos: `package.json` propio, `tsconfig.json` , y un `typecheck` que debía pasar antes de ejecutar. Así el script quedó reproducible y con un control de calidad sobre el código generado.

Categoría del supuesto	Casos	Coinciden
Éxito no documentado (se asumió 200)	7	7
Control (resultado documentado)	2	2

Seguridad (se asumió 401)	5	0
Campo requerido faltante	2	0
Valor fuera de enum	3	0
Tipo de dato incorrecto	2	0
Id inválido	3	0
Id int64 máximo	1	No ejecutado

Revisión crítica del reporte de la IA:

- **El conteo estaba inflado.** PET-010 no se ejecutó (otro usuario ya tenía una mascota con ese id, y el script correctamente no la tocó), pero se sumaba como "no coincide". La cifra correcta era 15 de 24 casos ejecutados.
- **Su recomendación final era peligrosa.** El agente sugirió "actualizar el status esperado a lo que la API realmente hace". Para los casos de seguridad, eso significaría escribir un test que certifica como correcto que cualquiera pueda borrar datos sin autenticarse. Esa recomendación se descartó, y en el prompt siguiente se prohibió explícitamente.

Iteración 3: clasificación con criterios propios (prompt 03)

Definí cinco clasificaciones y le pedí al agente aplicarlas de forma consistente a los 145 casos:

- **Automatizar:** resultado documentado y cumplido, o éxito verificado.
- **Automatizar con ajuste:** caso válido que necesita un cambio (por ejemplo, ids únicos en vez de fijos).
- **Hallazgo:** la API contradice un código documentado, responde 5xx ante input inválido o expone información interna.
- **Pregunta al negocio:** el comportamiento no está definido en la spec y la API no aplica la regla asumida. **Nunca se cambia el esperado al comportamiento real.**
- **Descartar:** duplicado, no determinista en una API compartida o de muy bajo valor.

Los casos no ejecutados se clasificaron por analogía con los verificados de la misma categoría.

El agente declaró por su cuenta una regla de precedencia para los casos que cumplen más de un criterio, y advirtió que la base era débil: **120 de 145 casos (83%) estaban clasificados por analogía**.

Revisión crítica:

- **Solo 6 de 39 hallazgos estaban comprobados.** Reportar 39 defectos con 6 confirmados sería un error.
- **USR-033 y USR-034** (login con credenciales inválidas) quedaron como "Automatizar" porque la spec documenta 400. Si la API no valida autenticación en ningún endpoint, era probable que el login tampoco lo hiciera. La analogía no se había aplicado.
- **Inconsistencia de criterio:** PET-003 (POST sin `name`) estaba como Pregunta y PET-020 (PUT sin `name`) como Hallazgo, aunque ambos endpoints documentan el mismo código de error.
- **El razonamiento quedó fuera del repositorio:** el script con la clasificación caso a caso estaba en una carpeta temporal.

Iteración 4: confirmar y corregir (prompt 04)

Agregué un caso representativo por cada patrón de analogía, más USR-033 y USR-034, y pedí reclasificar en cascada lo que dependiera de cada resultado.

La analogía falló dos veces:

- **Login:** acepta **cualquier** credencial: password incorrecta, usuario inexistente y username faltante devuelven 200 con una sesión. USR-033 y USR-034 pasaron de "Automatizar" a "Hallazgo". Es el hallazgo más grave.
- **Fecha inválida (STO-016):** se esperaba un 500, como en los demás errores de deserialización. La API respondió 200 y **guardó otra fecha**: `2026-13-45` quedó como `2027-02-14`. Es corrupción silenciosa de datos, peor que un error visible.

También se corrigió la inconsistencia con una regla explícita: si la spec declara la restricción **y** documenta el código de error, pero la API no la aplica, es Hallazgo; si falta cualquiera de las dos, es Pregunta. El agente extendió esta regla a PET-007 (enum) sin que se lo pidiera, y lo declaró en el resumen.

Efecto de la segunda verificación: los hallazgos verificados pasaron de 6 a 18 con solo 9 ejecuciones adicionales, y el script de clasificación quedó versionado en `scripts/build-evaluacion.mjs`.

Iteración 5: automatización (prompt 05)

Le pedí al agente automatizar casos "Automatizar" no cubiertos y cuatro hallazgos verificados dentro del framework de la Parte 1, con reglas que vienen de errores reales de este proyecto: `await` en todas las aserciones asíncronas, ids únicos y cambios solo aditivos en `src/`.

Los hallazgos también se automatizaron, con `test.fail()`: el test valida lo que dice la spec, y Playwright lo registra como falla esperada. Mientras el defecto exista, el pipeline queda en verde, y cuando se corrija, el test avisa para actualizarlo.

Resultado: `npm run test:stg` pasó **16 de 16**: los 4 tests existentes, 8 nuevos y 4 hallazgos que fallan como se espera. El agente verificó con un reporte JSON que cada hallazgo fallara **en la aserción de la spec y no en el setup**, porque un `test.fail()` que falla por otra razón también aparecería en verde.

Decisiones del agente que revisé:

- **STO-028** verifica el borrado de una orden con un segundo DELETE en vez de un GET. Un GET con 404 habría pasado **gracias al bug** STO-023, no porque la orden se borró. Además, el agente eliminó una aserción que había escrito sin verificar.
- `updateWithForm` sobrescribe el `Content-Type: application/json` global del framework. Sin eso, el form se habría enviado mal etiquetado.
- **Soft assertions dentro de `test.fail()`:** si se corrige solo uno de los dos problemas de PET-027 o STO-016, el test seguirá marcado como falla esperada. Es aceptable, pero es una limitación a conocer.

---

4. Hallazgos principales

ID	Request	Spec	Respuesta real	Impacto
USR-033 / USR-034	`GET /user/login` con credenciales inválidas	400	**200** y una sesión	**Crítico.** El login acepta cualquier credencial.
PET-047	`DELETE /pet/{id}` sin credenciales	Declara OAuth2	**200**, la mascota se borra	Cualquier cliente anónimo puede borrar datos (Pregunta al negocio).

PET-027 / PET-044	`GET` y `DELETE /pet/abc`	400	**404** con `java.lang.NumberFormatException`	Desvío de la spec y exposición de detalles internos.
STO-016	Orden con `shipDate: "2026-13-45"`	400	**200**, guarda `2027-02-14`	Corrupción silenciosa de datos.
PET-006 / STO-011 / USR-004 / PET-005	Tipo incorrecto o JSON mal formado	400 / 405	**500** "something bad happened"	Errores de servidor ante input del cliente.
PET-003 / PET-007 / STO-010	Requerido faltante o valor fuera de enum	400 / 405	**200**, se guarda	La API no aplica su propio contrato.
STO-022 / STO-023	`GET /store/order/0` y `/11`	400 (rango 1 a 10)	**404**	No valida el rango documentado.

El inventario (`GET /store/inventory`) confirma el problema de validación a escala: contiene claves de status como `"AAA"` , `"null"` y `"avalible"` , guardadas por otros usuarios.

---

5. Respuestas

5.1 ¿Cuáles de los casos generados son útiles?

58 de 145 (40%) son útiles como tests de regresión (40 directos y 18 con ajuste), y **79 más (54%) son útiles como análisis**, aunque no como tests: 46 hallazgos y 33 preguntas al negocio. Solo 8 (6%) se descartaron.

La utilidad dependió del tipo de caso, y eso solo se supo al ejecutar:

- **Los casos positivos son confiables:** los 7 supuestos de éxito verificados se cumplieron.
- **Los casos negativos y de seguridad son útiles como ideas, no como esperados:** ninguno de sus supuestos se cumplió en la API real. Su valor está en revelar dónde la implementación se aparta de la spec.
- **Los casos que asumen datos fijos no sirven en una API compartida** (ids 1 a 10, conteos globales del inventario), aunque sean correctos según la spec.
- **La lista de ambigüedades fue tan útil como los casos.** Detectó, entre otras cosas, que la spec usa 405 para errores de validación, que nunca documenta 401 ni 403, que los rangos de `orderId` son inconsistentes entre POST, GET y DELETE, que `findByStatus` se contradice (`required` con `default`) y que `createWithArray` y `createWithList` son idénticos.

5.2 ¿Cuáles destacarías y por qué?

- **USR-033 y USR-034 (login con credenciales inválidas):** son los más valiosos porque revelaron el defecto más grave, y porque la propia IA los había clasificado mal. Demuestran que la clasificación por analogía no reemplaza la ejecución.
- **PET-027 (id no numérico):** en un solo caso aparecen dos problemas, el desvío del código documentado y la exposición de una excepción interna, que en un sistema real es información útil para un atacante.

- **STO-016 (fecha inválida):** el resultado fue distinto a lo que predecían tanto la spec como la analogía. La API no falla: guarda otro dato. Los defectos silenciosos son los más difíciles de detectar en producción.
- **PET-047 (DELETE sin credenciales):** no es un defecto confirmado, porque la spec no define el rechazo, pero es la pregunta de negocio con mayor riesgo.
- **PET-010 (id int64 máximo):** anticipa un problema técnico de la automatización, no de la API. JavaScript pierde precisión por encima de 2^53, así que estos ids deben compararse como string o BigInt.

5.3 ¿Cómo automatizamos los casos generados rápidamente usando IA?

Con un flujo en el que la IA hace el trabajo repetitivo y una persona toma las decisiones:

1. **Generar desde la spec real**, en un contexto limpio, con un formato estructurado y una columna de supuestos.
2. **Verificar una muestra contra el sistema real** con un script generado por la IA, eligiendo al menos un caso por tipo de supuesto.
3. **Clasificar con criterios definidos por una persona**, que la IA aplica de forma consistente a todos los casos. Las reglas críticas se escriben de forma explícita: por ejemplo, "nunca cambies el esperado al comportamiento real en casos de seguridad".
4. **Generar los tests con un agente que conoce el framework.** Al darle acceso a los clients, fixtures y factories existentes, el código sigue las convenciones del proyecto y se puede mantener. En este caso, la IA también extendió el framework con un recurso nuevo (User) siguiendo el mismo patrón.
5. **Automatizar los hallazgos con** `test.fail()`, para que los defectos queden documentados y vigilados en el pipeline.
6. **Revisar el código generado como un pull request**, antes de integrarlo.
7. **Ejecutar en CI.** Los tests nuevos corren en el pipeline de la Parte 4 sin cambios adicionales.

En este proyecto, el flujo completo (generación, dos verificaciones, clasificación y 12 tests integrados) tomó una tarde. El tiempo ahorrado está en la escritura; el tiempo invertido está en verificar y decidir, que es donde la IA no puede reemplazar el criterio.

---

6. Revisión manual

Muestra de casos revisados por mí, con mi decisión sobre la clasificación de la IA.

Caso	Clasificación IA	¿De acuerdo?	Por qué
PET-007	Hallazgo (regla extendida por la IA)	[COMPLETAR]	[COMPLETAR]
[ID]	Automatizar		
[ID]	Automatizar		
[ID]	Automatizar		
[ID]	Automatizar		
[ID]	Automatizar		
[ID]	Pregunta al negocio		
[ID]	Pregunta al negocio		
[ID]	Pregunta al negocio		

[ID]	Pregunta al negocio		
[ID]	Pregunta al negocio		

---

7. Lecciones y limitaciones

Lecciones:

- **La analogía sirve para priorizar, no para concluir.** Falló en dos de los patrones que se pusieron a prueba.
- **Las recomendaciones de la IA también se evalúan.** La sugerencia de alinear los esperados con el comportamiento real habría producido tests que certifican defectos de seguridad.
- **Los prompts mejoran con cada iteración.** Las reglas de los prompts 03 a 05 nacieron de errores observados en las salidas anteriores.
- **La reproducibilidad es parte del entregable.** Los scripts de verificación y clasificación quedaron en el repositorio, para que cualquiera pueda ver cómo se llegó a cada resultado.

Limitaciones:

- **111 de 145 casos (77%) siguen clasificados por analogía.** Cada patrón tiene al menos un caso verificado, salvo el overflow numérico (PET-011, STO-015, USR-005).
- **La API es pública y compartida:** los resultados pueden variar entre ejecuciones, y algunos casos (PET-010) no se pudieron ejecutar sin tocar datos ajenos.
- **Las preguntas al negocio requieren una respuesta** antes de convertirse en tests.

---

8. Archivos entregados

Archivo	Contenido
`inputs/swagger.json`	Especificación usada como única fuente
`prompts/`	Todos los prompts, en orden
`outputs/casos-*-raw.csv`	Salida original del LLM, sin modificar
`outputs/ambiguedades-*.md`	Ambigüedades de la spec detectadas por el LLM
`outputs/verificacion-api.csv`	Resultado de las 34 ejecuciones contra la API real
`outputs/evaluacion-casos.csv`	Clasificación de los 145 casos
`outputs/evaluacion-resumen.md`	Totales, hallazgos con evidencia, preguntas y correcciones
`scripts/verify-assumptions.ts`	Script de verificación (`npm run verify`)
`scripts/build-evaluacion.mjs`	Script de clasificación (`node scripts/build-evaluacion.mjs`)
`Parte 1/tests/ai-generated/`	Tests generados con IA, integrados al framework y al pipeline

---

Anexo: prompts utilizados

Prompt 00: restricciones para Claude Code

```
Usa únicamente el archivo `Parte 5B/inputs/swagger.json` como fuente. No leas otros
archivos del repositorio.
Utiliza el prompt `Parte 5B/prompts/01-generar-casos.md`.
Guarda la tabla en `Parte 5B/outputs/casos-[tag]-raw.csv` y la lista de ambigüedades en
`Parte 5B/outputs/ambiguedades-[tag].md`.
```

Prompt 01: generación de casos

```
## Rol
Eres un QA Engineer senior especializado en testing de APIs REST.

## Contexto
Adjunto la especificación Swagger 2.0 de la API Petstore (swagger.json).
- Es una API pública de demostración, compartida por muchos usuarios y sin aislamiento de
datos.
- Los casos se van a automatizar con Playwright + TypeScript.

## Tarea
Deriva los casos de prueba de TODOS los endpoints con el tag [pet | store | user].
Para cada endpoint cubre, cuando aplique:
- Positivos: flujo correcto (happy path).
- Negativos: parámetros faltantes, inválidos o de tipo incorrecto, recursos inexistentes,
body mal formado.
- Borde: límites de los tipos de dato, strings vacíos, ids extremos, valores fuera del
enum.
- Seguridad: comportamiento con y sin la autenticación declarada en la spec.

## Reglas
1. Básate solo en la especificación. No inventes endpoints, campos ni códigos de respuesta.
2. Si el resultado esperado de un caso no está documentado en la spec, indícalo en la
columna "Supuesto" con lo que asumiste.
3. El resultado esperado debe ser verificable: status code y qué validar en el body.
4. No repitas casos equivalentes.
5. No uses punto y coma (;) dentro de los campos.

## Formato de salida
Responde con una tabla CSV, usando punto y coma (;) como separador, con estas columnas:
ID;Endpoint;Método;Tipo;Título;Precondiciones;Datos de entrada;Resultado
esperado;Prioridad;Supuesto

- ID: PET-001, STO-001 o USR-001 según el tag.
- Tipo: Positivo, Negativo, Borde o Seguridad.
- Prioridad: Alta, Media o Baja.

Después de la tabla, agrega una lista breve de ambigüedades o vacíos que detectaste en la
especificación.
```

Prompt 02 (versión 2): verificación de supuestos

Cambio respecto a la versión 1: se agregó la sección "Prerrequisitos", porque el script no tenía sus dependencias declaradas y el editor marcaba errores de tipos.

```
## Objetivo
Contrastar los resultados esperados de los casos generados con el comportamiento real de la API Petstore, para evaluar críticamente qué supuestos se cumplen.

## Fuentes
- Casos: `Parte 5B/outputs/casos-pet-raw.csv`, `casos-store-raw.csv` y `casos-user-raw.csv`.
- No leas otros archivos del repositorio.

## Selección de la muestra
Elige entre 20 y 25 casos que cubran al menos uno de cada tipo:
- Respuesta exitosa no documentada en la spec (se asumió 200).
- Seguridad (se asumió 401 o 403).
- Campo requerido faltante.
- Valor fuera de enum.
- Tipo de dato incorrecto.
- Id no numérico, negativo o fuera de rango.
- Id int64 máximo.
- Caso sin supuesto (resultado documentado en la spec), como control.

## Prerrequisitos
- Node.js 18 o superior (necesario para `fetch` nativo y `AbortSignal.timeout`).
- `Parte 5B/` debe tener su propio `package.json` con estas devDependencies: `tsx`, `typescript` y `@types/node`. Agrega los scripts `verify` (`tsx scripts/verify-assumptions.ts`) y `typecheck` (`tsc --noEmit`). Si no existe, créalo.
- `Parte 5B/tsconfig.json` con `"types": ["node"]`, `"lib": ["ES2022", "DOM"]`, `"strict": true`, `"noEmit": true` e `include` limitado a `scripts/**/*.ts`.
- Instalación desde `Parte 5B/`: `npm install`.
- Antes de ejecutar, `npm run typecheck` debe terminar sin errores.

## Script
- Un solo archivo: `Parte 5B/scripts/verify-assumptions.ts`, ejecutable con `npm run verify` (desde `Parte 5B/`).
- Usa `fetch` nativo de Node. Base URL: `https://petstore.swagger.io/v2`.
- No envíes credenciales, para observar si la API exige autenticación.
- La API es pública y compartida: usa ids y usernames únicos (timestamp + random), crea los datos que cada caso necesite y no modifiques ni elimines datos que no hayas creado.
- Ejecuta los casos de forma secuencial, para no sobrecargar el servicio.

## Salida
Guarda `Parte 5B/outputs/verificacion-api.csv` (separador `;`) con las columnas:
ID;Tipo;Request;Status esperado;Status real;Coincide;Extracto del body
- El extracto del body debe tener como máximo 200 caracteres.
Al final, imprime en consola un resumen: total de casos, cuántos coinciden y cuántos no, agrupado por tipo.

No modifiques ningún otro archivo, salvo `Parte 5B/package.json` y `Parte 5B/tsconfig.json` si hace falta crearlos.
```

Prompt 03: clasificación de casos

```
## Objetivo
Consolidar los casos generados en una planilla de evaluación, aplicando los criterios
definidos abajo.

## Fuentes
- `Parte 5B/outputs/casos-pet-raw.csv`, `casos-store-raw.csv`, `casos-user-raw.csv`
- `Parte 5B/outputs/verificacion-api.csv`
- `Parte 5B/inputs/swagger.json`
No leas otros archivos.

## Criterios de clasificación
Asigna a cada caso UNA de estas clasificaciones:
- Automatizar: el resultado esperado está documentado en la spec y la API lo cumple, o es
un caso Positivo cuyo supuesto de éxito fue verificado.
- Automatizar con ajuste: el caso es válido, pero necesita un cambio para ser confiable o
correcto. Indica el ajuste concreto.
- Hallazgo: la API contradice un código documentado en la spec, responde 5xx ante input
inválido o expone información interna.
- Pregunta al negocio: el comportamiento esperado no está definido en la spec y la API no
aplica la regla asumida. No se automatiza hasta que alguien defina el comportamiento
correcto. NUNCA cambies el esperado al comportamiento real en estos casos.
- Descartar: duplicado, no determinista en una API compartida o de muy bajo valor. Indica
el motivo.

## Casos no verificados
Para los casos que no están en `verificacion-api.csv`, aplica el criterio por analogía con
los casos verificados de la misma categoría y marca la columna Verificado como "Por
analogía". PET-010 se marca como "No ejecutado".

## Salida
1. `Parte 5B/outputs/evaluacion-casos.csv` (separador `;`) con las columnas:
   ID;Endpoint;Método;Tipo;Título;Prioridad;Verificado;Status esperado (IA);Status
real;Clasificación;Justificación
2. `Parte 5B/outputs/evaluacion-resumen.md` con los totales por clasificación y tag, los
hallazgos con evidencia y las preguntas al negocio agrupadas por tema.

No modifiques ningún otro archivo.
```

Prompt 04: confirmación y corrección

```
## Objetivo
Confirmar contra la API real los patrones clasificados por analogía y corregir la
evaluación con la nueva evidencia.

## Tareas
1. Agrega al script existente `verify-assumptions.ts` un caso representativo por cada
patrón de "Hallazgos por analogía": PET-005, USR-004, STO-016, PET-044, STO-022, PET-066 y
USR-035. Agrega también USR-033 y USR-034, que hoy están como "Automatizar" sin verificar.
```

2. Mantén las mismas reglas: sin credenciales, ids únicos, ejecución secuencial y sin tocar datos ajenos.
3. Ejecuta con `npm run verify` y actualiza `verificacion-api.csv`.
4. Actualiza la evaluación. Si un resultado contradice la analogía, reclasifica ese caso y todos los que dependían de él, y explica cada cambio en una sección "Correcciones tras la segunda verificación".
5. Corrección de consistencia: PET-003 y PET-004 están como "Pregunta al negocio", pero POST /pet documenta 405 Invalid input y el caso equivalente en PUT (PET-020) está como "Hallazgo". Aplica el mismo criterio a todos los casos de campo requerido faltante y explica la regla usada.
6. Guarda el script de clasificación en `Parte 5B/scripts/build-evaluacion.mjs`, para que la evaluación sea reproducible.

No modifiques otros archivos.

Prompt 05: automatización en el framework

```
## Objetivo
Automatizar una selección de los casos evaluados dentro del framework existente de la Parte 1, respetando sus convenciones.

## Contexto
- Lee `Parte 1/` para entender el framework: clients, types, factories, fixtures, tests y playwright.config.ts.
- Casos: `Parte 5B/outputs/evaluacion-casos.csv` y la evidencia en `Parte 5B/outputs/verificacion-api.csv`.

## Selección
1. Entre 6 y 8 casos clasificados como Automatizar que no estén cubiertos por los tests existentes. Incluye al menos uno del tag user.
2. Los hallazgos verificados PET-027, USR-033, STO-016 y PET-006.

## Reglas
- Reutiliza los clients, factories y fixtures existentes. Si falta un recurso, créalo con el mismo patrón.
- Los cambios en `src/` deben ser solo aditivos. No modifiques los tests existentes.
- Todas las aserciones asíncronas llevan `await`.
- Usa ids y usernames únicos. Cada test crea sus propios datos y es independiente.
- Usa `test.step` con nombres en español. El título de cada test empieza con el ID del caso.
- Hallazgos: el test valida lo que dice la spec, no el comportamiento actual. Márcalo con `test.fail()` y una anotación de tipo `issue` que describa el bug y la evidencia.
- Crea los tests en `Parte 1/tests/ai-generated/`.

## Validación
- Ejecuta `npm run test:stg` desde `Parte 1/`. Todos los tests deben pasar.
- Si un test falla de forma inesperada, analiza la causa antes de cambiarlo. No ajustes el esperado de un hallazgo para que pase.

## Salida
```

```
Lista de archivos creados o modificados, tabla con cada caso y su resultado, y las
decisiones que deba revisar.
```